Department of Statistics and Computer Science
Faculty of Science
University of Peradeniya

# A Multivariate Statistical Analysis of AI-Driven Job Market Data Using PCA, Factor Analysis, and Discriminant Analysis

DSC 4033 / STA 4053 - Multivariate Methods II
Mini Project Report

N.Tharsika
S/20/851
17th April 2026

# Table of Contents

1. Introduction
2. Problem Formulation
3. Dataset Description and Pre-processing
4. Methods used
5. Methodology
   5.1 Principal Component Analysis
   5.2 Factor Analysis
   5.3 Discriminant Analysis
      5.3.1 Linear Discriminant Analysis
      5.3.2 Quadratic Discriminant Analysis
6. Results and Discussion
7. Conclusion
8. Recommendations
9. References
10. Appendices

# 1. INTRODUCTION

The rapid advancement of Artificial Intelligence (AI) and Machine Learning (ML) technologies has significantly transformed the global job market, leading to the emergence of new job roles and evolving skill requirements. Positions such as LLM Engineer, AI Agent Developer, and Prompt Engineer reflect the growing complexity and specialization within the AI domain. Understanding how job characteristics, compensation, and market demand interact is essential for informed decision-making by job seekers, organizations, and policymakers.

This study utilizes a dataset comprising 1,500 AI and ML job postings from 2025–2026 across multiple countries and industries. The dataset includes variables related to salaries, experience levels, company size, demand metrics, and job benefits, providing a comprehensive view of the AI job market.

Given the multivariate nature of the dataset, advanced statistical techniques are required to uncover underlying patterns and relationships. Therefore, this study applies Principal Component Analysis (PCA), Factor Analysis (FA), and Discriminant Analysis to explore the structure of the data and extract meaningful insights.

The main objectives of this study are to identify key factors influencing salaries and demand, reduce dimensionality, and classify job roles based on relevant characteristics.

# 2. PROBLEM FORMULATION

The AI job market is characterized by complex relationships among multiple variables, including salary, demand, experience, and organizational factors. Traditional analytical methods are insufficient to capture these interdependencies.

One major challenge is the high dimensionality of the dataset, where several variables are correlated, particularly salary-related variables. Additionally, demand-related variables and job quality indicators may reflect underlying latent constructs that are not directly observable.

This study seeks to address the following research questions:

- What latent factors explain variations in salaries, demand, and job quality?
- Can the dataset be reduced to a smaller set of components without losing significant information?
- Can job roles or salary tiers be classified using quantitative variables?
- How do different multivariate techniques compare in revealing data structure?

To address these questions, PCA is used for dimensionality reduction, FA for identifying latent constructs, and Discriminant Analysis for classification.

# 3. DATASET DESCRIPTION AND PRE-PROCESSING

## Dataset Description

- Source of Dataset: The dataset was obtained from a publicly available source on Kaggle and, compiled using multiple industry reports, including global salary benchmarks, recruitment reports, and workforce insights. These sources include platforms such as LinkedIn Workforce Insights, Glassdoor, and other AI/ML salary reports.

- Number of Observations and Variables:
  The dataset contains: **1,500 observations**, representing individual job postings
  **25 variables**, describing various aspects of AI job roles

- Types of Variables:
  The dataset includes a mix of categorical and numerical variables:

  1. Categorical Variables
     `job_title, job_category, experience_level, industry, education_required, city, country, remote_work, company_size, salary_tier`

  2. Numerical Variables
     `years_of_experience, annual_salary_usd, demand_score salary_min_usd, salary_max_usd, benefits_score ai_salary_premium_pct, demand_growth_yoy_pct`

  3. Binary Variables (Dummy Variables)
     `is_senior, is_remote_friendly, is_llm_role`

## Pre-processing

Prior to analysis, the dataset was examined and pre-processed to ensure its suitability for multivariate techniques.

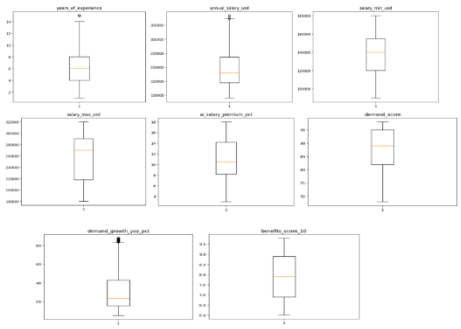
### 3.1 Handling Missing Values

A check for missing values indicated that all variables were complete, with no missing observations detected (see Appendix A1). Therefore, no imputation or data removal procedures were required.

### 3.2 Variable Selection

Certain variables were excluded from the analysis due to their limited analytical value. Specifically, `job_id`, `job_title`, `city`, and `required_skills` were removed, as they represent identifiers, high-cardinality categorical data, or unstructured text. Additionally, `posting_year` and `posting_month` were excluded due to limited variability and lack of relevance to the study objectives.The analysis focused on key numerical variables related to salary, demand, and job characteristics.

### 3.3 Outlier Detection

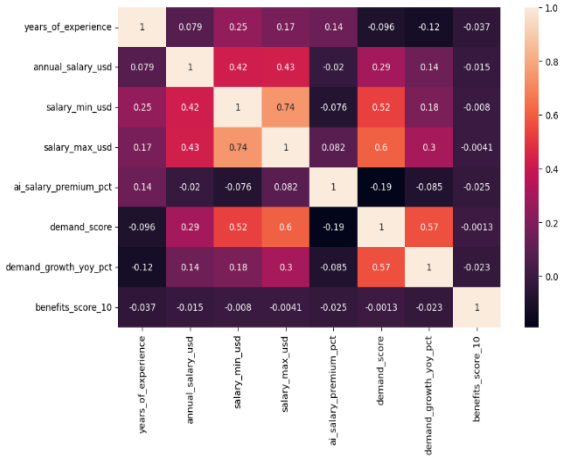


Outliers were assessed using boxplots (Appendix A4). Although some extreme values were observed, these were consistent with real-world scenarios, such as high-paying senior roles.

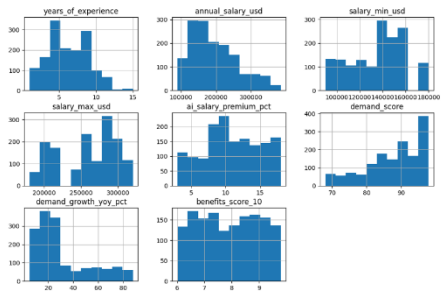
Therefore, outliers were retained to preserve the integrity and realism of the dataset.

### 3.4 Exploratory Data Analysis

Descriptive statistics were computed to summarize the dataset (see Appendix A2). The results indicate substantial variability in salary-related variables, reflecting differences across job roles and experience levels.




Correlation analysis (see Appendix A5) revealed strong positive relationships among salary variables, indicating redundancy and justifying the use of dimensionality reduction techniques. Moderate correlations were observed among demand-related variables, suggesting the presence of latent structures.




Histograms (see Appendix A3) were used to examine the distribution of variables, providing further insight into data characteristics.

### 3.5 Standardization

Since variables were measured on different scales, standardization was applied to ensure comparability. Each numerical variable was transformed to have a mean of zero and a standard deviation of one (see Appendix A6).

Standardization is essential for PCA and Factor Analysis, as these methods are sensitive to the variance of the variables. Without standardization, variables with larger scales would dominate the analysis.

### 3.6 Data Transformation and Encoding

Categorical variables such as job category, company size, and remote work type were encoded into numerical formats to facilitate analysis. Binary variables (e.g., remote-friendly, senior roles) were retained as indicator variables (default with the dataset).

# 4. METHODS USED

1. Principal Component Analysis (PCA)
2. Factor Analysis
3. Discriminant Analysis
   I. Linear Discriminant Analysis (LDA)
   II. Quadratic Discriminant Analysis (QDA)

# 5. METHODOLOGY

## 5.1 PRINCIPAL COMPONENT ANALYSIS (PCA)

Principal Component Analysis (PCA) was applied to reduce the dimensionality of the dataset and to address multicollinearity among numerical variables. The correlation matrix (see Appendix A5) reveals strong positive correlations among salary-related variables, particularly between `salary_min_usd` and `salary_max_usd` (0.74), as well as moderate correlations with `annual_salary_usd`.

Additionally, demand-related variables such as `demand_growth_yoy_pct` and `demand_score` exhibit moderate correlation (0.57), suggesting the presence of underlying latent structures. These interrelationships indicate redundancy in the dataset, making PCA an appropriate method to summarize the information into a smaller set of uncorrelated components.

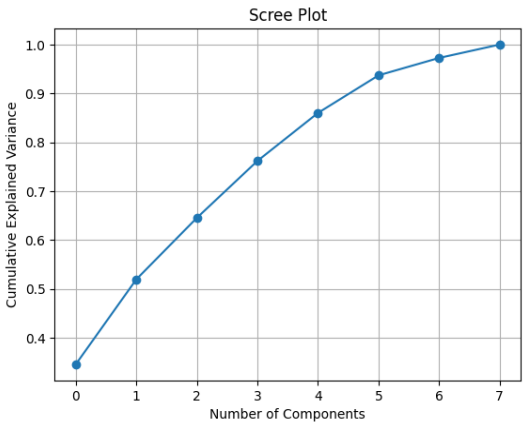
### 5.1.1 Assumption Checking

The suitability of the dataset for PCA was assessed based on the following:

- Linearity: The correlation matrix indicates several linear relationships among variables, particularly among salary and demand variables, satisfying the linearity assumption.
- Adequate Correlation: The presence of moderate to strong correlations (e.g., 0.74 between salary variables and 0.57 between demand variables) confirms that PCA is appropriate.
- Sample Size Adequacy: The dataset contains 1,500 observations, which is sufficiently large for reliable PCA results.
- Standardization: All numerical variables were standardized prior to analysis to ensure equal contribution (see Appendix A6), as variables were measured on different scales.

### 5.1.2 Principal Component Analysis (PCA) Results

**Component Selection (Scree Plot Analysis)**

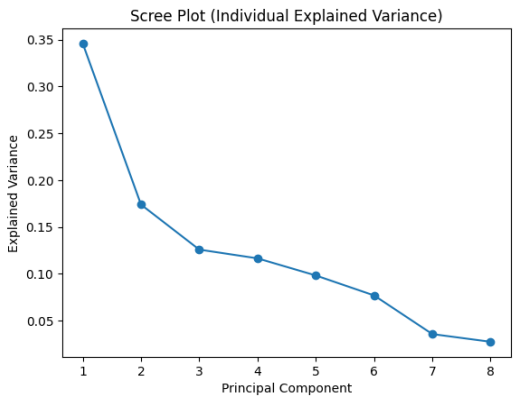



- The scree plot illustrates the cumulative explained variance against the number of principal components. A sharp increase in explained variance is observed for the first few components, followed by a gradual flattening of the curve.

- The first principal component explains approximately **34.59%** of the total variance, while the first two components together account for approximately **51.98%**. The cumulative explained variance reaches approximately **76.20%** with four components and **86.02%** with five components.

- An "elbow" is observed around the fourth to fifth component, indicating diminishing returns beyond this point. Therefore, **five principal components were retained**, as they capture a substantial proportion of the total variance while achieving effective dimensionality reduction.

**Explained Variance**




The distribution of explained variance across components indicates that the first principal component contributes the largest proportion of variability, suggesting a dominant underlying structure in the dataset.

Subsequent components contribute progressively smaller proportions of variance, confirming that much of the dataset's information is concentrated in the first few components. This pattern highlights the presence of redundancy among variables, particularly among salary-related variables.

### 5.1.3 Interpretation of Principal Components

Loadings:

| | PC1 | PC2 | PC3 | PC4 | PC5 |
|---|---|---|---|---|---|
| years_of_experience | 0.084234 | 0.633051 | 0.034292 | -0.204632 | -0.578763 |
| annual_salary_usd | 0.355658 | 0.160034 | 0.065965 | -0.148691 | 0.712273 |
| salary_min_usd | 0.497029 | 0.223852 | 0.093461 | -0.181276 | -0.040792 |
| salary_max_usd | 0.521510 | 0.179328 | 0.022048 | 0.108395 | 0.004172 |
| ai_salary_premium_pct | -0.062631 | 0.456171 | -0.181286 | 0.825429 | 0.155076 |
| demand_score | 0.489341 | -0.303648 | -0.042499 | 0.085783 | -0.149567 |
| demand_growth_yoy_pct | 0.322312 | -0.430506 | -0.197391 | 0.361658 | -0.323854 |
| benefits_score_10 | -0.011769 | -0.075751 | 0.954787 | 0.268175 | -0.068692 |

The principal components reveal meaningful structures within the AI job market dataset:

- **PC1 (Compensation and Market Strength):**
  The first principal component reflects overall compensation and market demand, with strong positive contributions from salary and demand-related variables.
- **PC2 (Experience vs Demand Trade-off):**
  The second component captures a contrast between experience and market demand, indicating that higher experience levels are associated with lower demand indicators.
- **PC3 (Job Benefits Factor):**
  The third component is dominated by benefits score, representing job quality and benefits as an independent dimension.
- **PC4 (AI Salary Premium Factor):**
  The fourth component reflects AI-specific salary premiums, capturing additional compensation associated with AI-related roles.
- **PC5 (Salary vs Experience Contrast):**
  The fifth component represents a contrast between salary and experience, suggesting that higher salaries may not always correspond to higher experience levels.

### 5.1.4 Summary of PCA Findings

The PCA results demonstrate that the dataset can be effectively reduced to five principal components while retaining approximately 86% of the total variance. The analysis indicates that the AI job market is primarily driven by compensation and market demand factors. In addition, distinct components capture job benefits, AI salary premiums, and trade-offs between experience, salary, and demand. Overall, the results highlight the multidimensional structure of the dataset, with clearly interpretable patterns underlying job characteristics.

## 5.2 FACTOR ANALYSIS (FA)

Factor Analysis (FA) was applied to identify underlying latent variables that explain the observed correlations among the numerical variables. While PCA focuses on variance reduction, FA is more suitable for uncovering hidden constructs that influence observed variables.

Given the presence of correlated variables, particularly among salary-related and demand-related variables. FA is appropriate for grouping these variables into meaningful factors such as compensation, market demand, and job quality. This allows for a deeper understanding of the structural relationships within the AI job market.

### 5.2.1 Assumption Checking

Before applying Factor Analysis, the suitability of the data was evaluated using the following tests:

1. **Kaiser-Meyer-Olkin (KMO) Test**
   The KMO test measures sampling adequacy and determines whether the dataset is suitable for factor analysis. A value greater than 0.6 is considered acceptable.

```
KMO: 0.6818683705817283
```

- The KMO value was found to be approximately `0.682`, indicating that the sample is adequate for factor analysis.

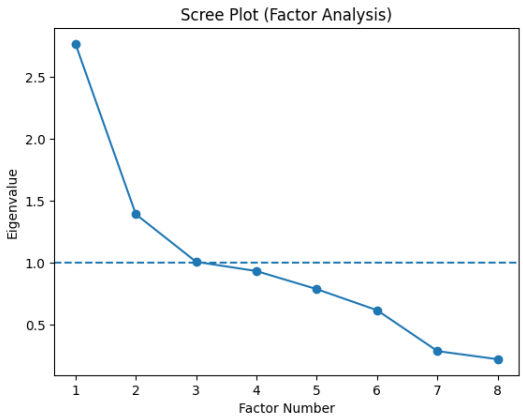
2. **Bartlett's Test of Sphericity**
   Bartlett's test checks whether the correlation matrix is significantly different from an identity matrix.

```
Bartlett Test p-value: 0.0
```

- Bartlett's test was statistically significant ($p < 0.05$), indicating that the variables are sufficiently correlated for factor analysis.

### 5.2.2 Factor Analysis Results

**Factor Extraction**




The number of factors was determined using the Kaiser criterion (eigenvalues > 1). The first three factors had eigenvalues of 2.767, 1.391, and 1.006, while all remaining factors had eigenvalues less than 1.

The scree plot also showed a clear decline after the third factor. Therefore, three factors were retained.

Additionally, the three-factor solution provided meaningful and interpretable factors.

### 5.2.3 Interpretation of Factors

| | Factor1 | Factor2 | Factor3 |
|---|---|---|---|
| years_of_experience | 0.293842 | -0.260300 | 0.170154 |
| annual_salary_usd | 0.445627 | 0.138939 | 0.035016 |
| salary_min_usd | 0.922094 | 0.115647 | -0.027706 |
| salary_max_usd | 0.778943 | 0.337688 | 0.228270 |
| ai_salary_premium_pct | -0.047478 | -0.121999 | 0.697184 |
| demand_score | 0.452598 | 0.813630 | -0.101749 |
| demand_growth_yoy_pct | 0.120943 | 0.623185 | -0.003594 |
| benefits_score_10 | -0.011453 | -0.002456 | -0.040764 |

**Factor 1 – Compensation Factor**
This factor has strong positive loadings on:
- `salary_min_usd` (0.922)
- `salary_max_usd` (0.779)
- `annual_salary_usd` (0.446)

Interpretation:
This factor represents the overall compensation structure, indicating that salary-related variables strongly influence this dimension. It captures variations in pay across different AI job roles.

**Factor 2 – Market Demand Factor**
This factor is strongly associated with:
- `demand_score` (0.814)
- `demand_growth_yoy_pct` (0.623)

Interpretation:
This factor represents market demand conditions, reflecting how demand levels and growth trends influence job opportunities in the AI job market.

**Factor 3 – Salary Premium / Specialized Skills Factor**
This factor shows a strong loading on:
- `ai_salary_premium_pct` (0.697)

Interpretation:

This factor represents the salary premium associated with AI-specific skills, indicating that certain specialized roles command higher compensation compared to traditional roles.

**Note on Benefits Variable**

- `benefits_score_10` has very low loadings

Interpretation:

The benefits variable does not load significantly on any factor, suggesting that job benefits form a weak or independent dimension in this dataset.

### 5.2.4 Summary of Factor Analysis

The Factor Analysis results indicate that the dataset can be effectively summarized into three key latent factors:

- Compensation
- Market Demand
- Salary Premium / Specialized Skills

These findings align with PCA results, reinforcing that salary and demand are the primary drivers of variation in the AI job market.

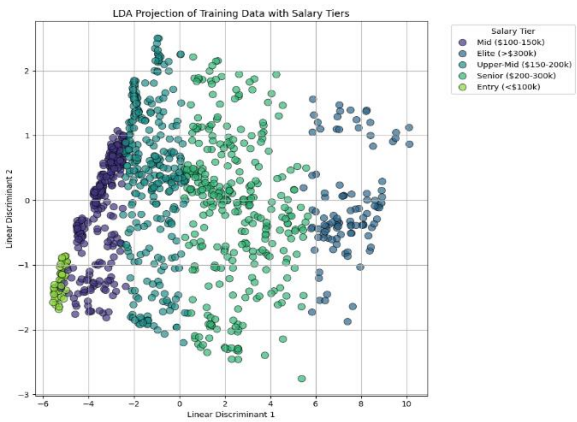
## 5.3 DISCRIMINANT ANALYSIS

### 5.3.1 Linear Discriminant Analysis (LDA)

Linear Discriminant Analysis (LDA) was applied as a classification technique to predict group membership based on numerical variables. In this study, LDA is used to classify job postings into different salary tiers using variables such as salary, demand, and experience.

LDA is appropriate because it reduces dimensionality while maximizing separation between predefined groups. It is also suitable for datasets with continuous predictors and relatively large sample sizes.

#### 5.3.1.1 Assumption Checking




**1. Multivariate Normality**

LDA assumes that predictor variables are approximately normally distributed within each group. Given the large sample size (n = 1500), this assumption is reasonably satisfied due to the Central Limit Theorem.

**2. Equality of Covariance Matrices (Box's M Test)**

Box's M test is used to determine whether covariance matrices are equal across groups.

Although formal testing such as Box's M test can be used to assess equality of covariance matrices, LDA is generally robust to moderate violations of this assumption, particularly with large sample sizes.

3. **Independence of Observations**

Each observation represents a unique job posting; therefore, observations can be considered independent.

#### 5.3.1.2 Implementation

LDA was applied using numerical predictor variables to classify job postings based on salary tier.

#### 5.3.1.3 Results and Interpretation

**Classification Accuracy**

The Linear Discriminant Analysis (LDA) model achieved an accuracy of **93.56%**, indicating a high level of classification performance. This suggests that the selected numerical variables are highly effective in distinguishing between different salary tiers in the AI job market.

**Confusion Matrix Analysis**

The confusion matrix is presented in Appendix.

Interpretation:

- The model correctly classified the majority of observations across all salary tiers.
- Very few misclassifications are observed, indicating strong separation between groups.

Key Observations:

- The **middle salary tiers** (e.g., third and fourth categories) show very high correct classification rates.
- Minor misclassification occurs between **adjacent salary tiers**, which is expected due to similar characteristics.
- The **highest and lowest salary tiers** are classified with high accuracy, indicating clear distinction in their features.

#### 5.3.1.4 Interpretation

The high classification accuracy demonstrates that variables such as salary structure, demand indicators, and experience effectively differentiate between salary tiers. The results indicate that the AI job market exhibits clear segmentation, with well-defined characteristics distinguishing different levels of compensation.

### 5.3.2 Quadratic Discriminant Analysis (QDA)

In addition to Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA) was also implemented to compare classification performance.

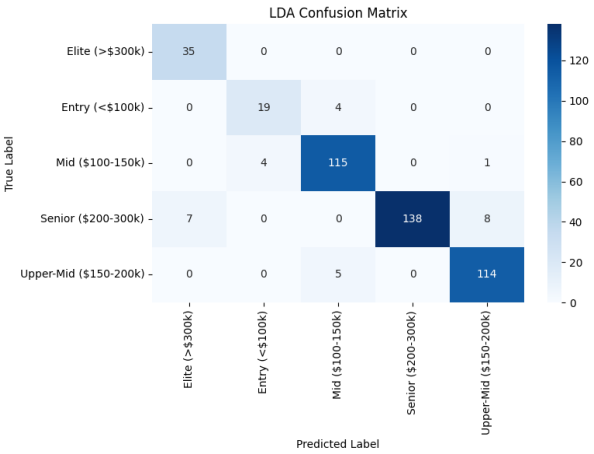
#### 5.3.2.1 Linear Discriminant Analysis (LDA) vs. Quadratic Discriminant Analysis (QDA)

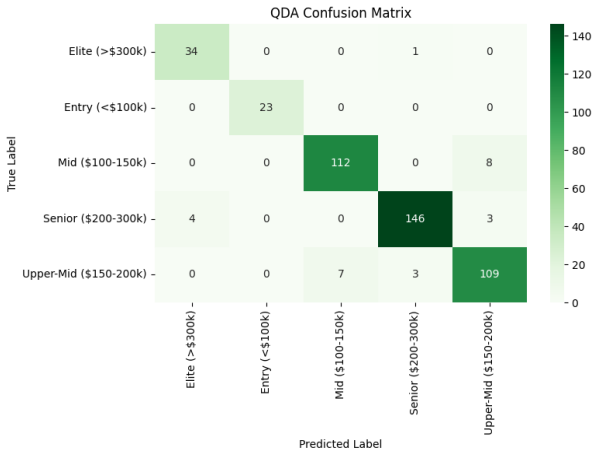
Both LDA and QDA are classification algorithms that aim to find a linear or quadratic boundary between classes. The key difference lies in their assumptions about the covariance matrices of the classes:

- **LDA (Linear Discriminant Analysis):** Assumes that all classes share the same covariance matrix. This simplifies the mo=del, leading to linear decision boundaries.
- **QDA (Quadratic Discriminant Analysis):** Does not assume equal covariance matrices for all classes. Instead, it allows each class to have its own covariance matrix, resulting in quadratic decision boundaries.

**Why Compare When Assumptions Differ?**
While LDA's assumption of equal covariance matrices is a strong one, it's still useful to compare its performance with QDA. If the true underlying data distributions violate this assumption, QDA is theoretically more flexible and should perform better.

The QDA model achieved an accuracy of **94.22%**, which is slightly higher than the LDA accuracy of **93.56%**.




In addition to accuracy, precision, recall, and F1-score were evaluated. Both models achieved a **weighted F1-score of 0.94**, demonstrating a strong balance between precision and recall across all salary tiers.

LDA Classification Report:

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| Elite (>$300k) | 0.83 | 1.00 | 0.91 | 35 |
| Entry (<$100k) | 0.83 | 0.83 | 0.83 | 23 |
| Mid ($100-150k) | 0.93 | 0.96 | 0.94 | 120 |
| Senior ($200-300k) | 1.00 | 0.90 | 0.95 | 153 |
| Upper-Mid ($150-200k) | 0.93 | 0.96 | 0.94 | 119 |
| accuracy | | | 0.94 | 450 |
| macro avg | 0.90 | 0.93 | 0.91 | 450 |
| weighted avg | 0.94 | 0.94 | 0.94 | 450 |

QDA Classification Report:

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| Elite (>$300k) | 0.89 | 0.97 | 0.93 | 35 |
| Entry (<$100k) | 1.00 | 1.00 | 1.00 | 23 |
| Mid ($100-150k) | 0.94 | 0.93 | 0.94 | 120 |
| Senior ($200-300k) | 0.97 | 0.95 | 0.96 | 153 |
| Upper-Mid ($150-200k) | 0.91 | 0.92 | 0.91 | 119 |
| accuracy | | | 0.94 | 450 |
| macro avg | 0.94 | 0.95 | 0.95 | 450 |
| weighted avg | 0.94 | 0.94 | 0.94 | 450 |

### 5.3.2.2 Interpretation
Although QDA showed a marginal improvement in classification accuracy, the difference between the two models is relatively small. This suggests that both models perform similarly in classifying salary tiers.

QDA allows for different covariance matrices across groups, making it more flexible than LDA. The slightly higher accuracy of QDA may indicate some variation in covariance structures among groups.

**Overall Summary**
However, LDA was preferred for final interpretation due to its simplicity, better interpretability, and robustness, particularly given the large sample size. Since the improvement in QDA accuracy is minimal, LDA provides a more stable and parsimonious model.

The results suggest that while covariance structures may differ slightly across groups, the assumption of equal covariance in LDA does not significantly affect classification performance.

## 6. RESULTS AND DISCUSSION

The application of Principal Component Analysis (PCA), Factor Analysis (FA), and Discriminant Analysis provided complementary insights into the AI job market dataset.
PCA revealed that a large proportion of the total variance is explained by the first few components, with salary-related variables contributing most significantly. This indicates that compensation is a key driver of variation across AI job roles.

Factor Analysis further simplified the data structure by identifying three meaningful latent factors: compensation, market demand, and salary premium. These factors provide a clearer interpretation of the underlying dimensions influencing job characteristics and confirm the patterns observed in PCA.

Discriminant Analysis demonstrated strong classification performance, with both LDA and QDA achieving high accuracy and F1-scores (approximately 94%). The results indicate that salary tiers are well-separated and can be effectively predicted using numerical variables such as salary, demand, and experience.

Overall, the findings consistently highlight that compensation and market demand are the dominant factors shaping the AI job market, and that these variables are highly effective in distinguishing between different job categories.

## 7. CONCLUSION

This study applied multivariate statistical techniques, including Principal Component Analysis (PCA), Factor Analysis (FA), and Discriminant Analysis, to analyse the AI job market dataset.

PCA revealed that salary-related variables contribute most to overall variation with five main principal components, while Factor Analysis identified three key latent factors: compensation, market demand, and salary premium. Discriminant Analysis demonstrated that salary tiers can be accurately classified using these variables, achieving over 93% accuracy.

Overall, the findings indicate that the AI job market is strongly driven by compensation structures and demand dynamics, with clear differentiation between job categories.

## 8. RECOMMENDATIONS

- Companies should focus on competitive salary structures to attract talent in high-demand AI roles.
- Job seekers should prioritize skills associated with high salary premiums, particularly in emerging AI areas.
- Organizations should monitor market demand trends to align hiring strategies effectively.
- Future studies could incorporate additional variables such as company performance or cost of living for deeper analysis.

## 9. REFERENCES


Data Source:
*https://www.kaggle.com/datasets/alitaqishah/ai-jobs-market-2025-2026-salaries*

- *https://uw.pressbooks.pub/appliedmultivariatestatistics/chapter/discriminant-analysis/*
- *Bartlett, M. S. (1947). Multivariate analysis. Supplement to the journal of the royal statistical society, 9(2), 176-197*
- *Backhaus, K., Erichson, B., Gensler, S., Weiber, R., Weiber, T. (2021). Multivariate analysis. Springer Books, 10(1), 973-8.*
- *Mardia, K. V., Kent, J. T., Taylor, C. C. (2024). Multivariate analysis. John Wiley Sons.*

# 10. APPENDICES

## APPENDIX A: DATA PREPARATION AND EDA OUTPUTS

### Appendix A1: Missing Values

```
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   job_id                 1500 non-null   object
 1   job_title              1500 non-null   object
 2   job_category           1500 non-null   object
 3   experience_level       1500 non-null   object
 4   years_of_experience    1500 non-null   int64
 5   education_required     1500 non-null   object
 6   annual_salary_usd      1500 non-null   float64
 7   salary_min_usd         1500 non-null   int64
 8   salary_max_usd         1500 non-null   int64
 9   city                   1500 non-null   object
 10  country                1500 non-null   object
 11  remote_work            1500 non-null   object
 12  company_size           1500 non-null   object
 13  industry               1500 non-null   object
 14  required_skills        1500 non-null   object
 15  ai_salary_premium_pct  1500 non-null   float64
 16  demand_score           1500 non-null   int64
 17  demand_growth_yoy_pct  1500 non-null   float64
 18  benefits_score_10      1500 non-null   float64
 19  posting_year           1500 non-null   int64
 20  posting_month          1500 non-null   int64
 21  is_senior              1500 non-null   int64
 22  is_remote_friendly     1500 non-null   int64
 23  is_llm_role            1500 non-null   int64
 24  salary_tier            1500 non-null   object
dtypes: float64(4), int64(9), object(12)
```

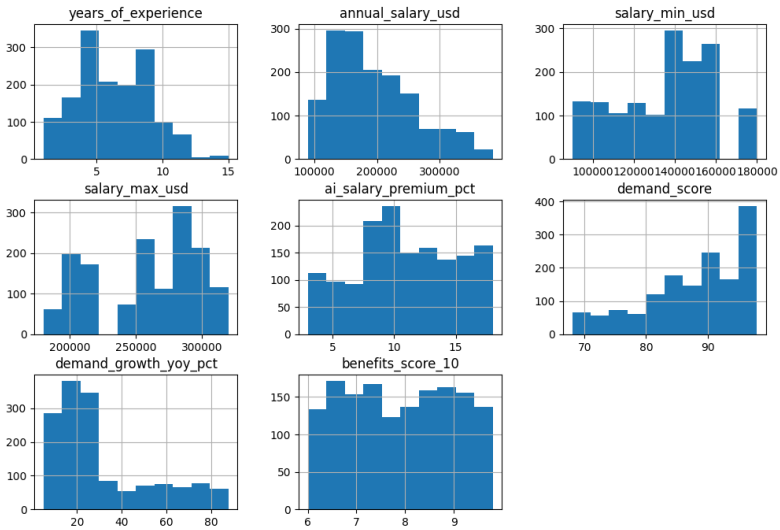
- It shows that there are no missing values in the dataset.

### Appendix A2: Descriptive Statistics

| | years_of_experience | annual_salary_usd | salary_min_usd | salary_max_usd | ai_salary_premium_pct | demand_score | demand_growth_yoy_pct | benefits_score_10 |
|---|---|---|---|---|---|---|---|---|
| **count** | 1500.000000 | 1500.000000 | 1500.000000 | 1500.000000 | 1500.000000 | 1500.000000 | 1500.000000 | 1500.000000 |
| **mean** | 6.216000 | 194892.000000 | 135448.666667 | 257537.333333 | 10.858200 | 87.523333 | 31.116333 | 7.897333 |
| **std** | 2.675216 | 66506.822013 | 24448.950878 | 39852.822207 | 4.029742 | 8.026315 | 22.046343 | 1.102846 |
| **min** | 1.000000 | 90000.000000 | 90000.000000 | 180000.000000 | 3.000000 | 68.000000 | 5.000000 | 6.000000 |
| **25%** | 4.000000 | 144750.000000 | 120000.000000 | 218000.000000 | 8.200000 | 82.000000 | 15.375000 | 6.900000 |
| **50%** | 6.000000 | 180000.000000 | 140000.000000 | 270000.000000 | 10.500000 | 89.000000 | 23.400000 | 7.900000 |
| **75%** | 8.000000 | 236250.000000 | 155000.000000 | 290000.000000 | 14.200000 | 95.000000 | 42.700000 | 8.900000 |
| **max** | 15.000000 | 384000.000000 | 180000.000000 | 320000.000000 | 18.000000 | 98.000000 | 87.800000 | 9.800000 |

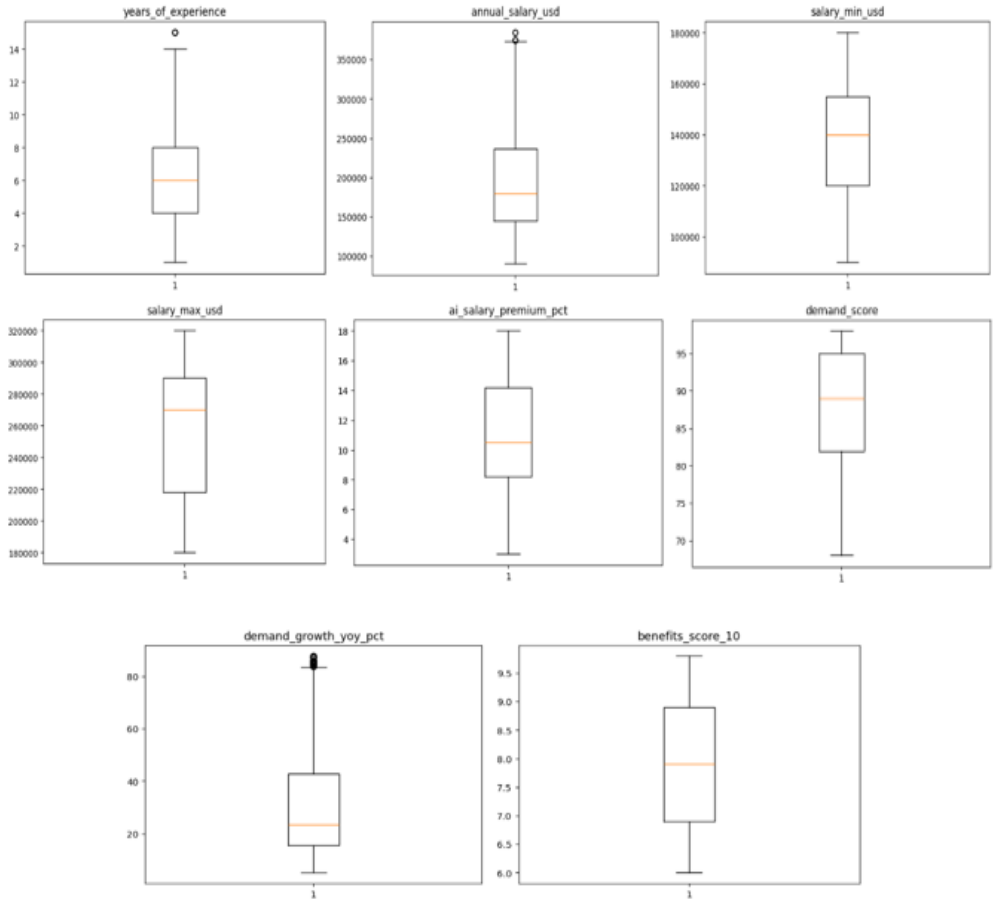
- This table presents summary statistics for numerical variables, including mean, standard deviation, and range.

### Appendix A3: Histograms

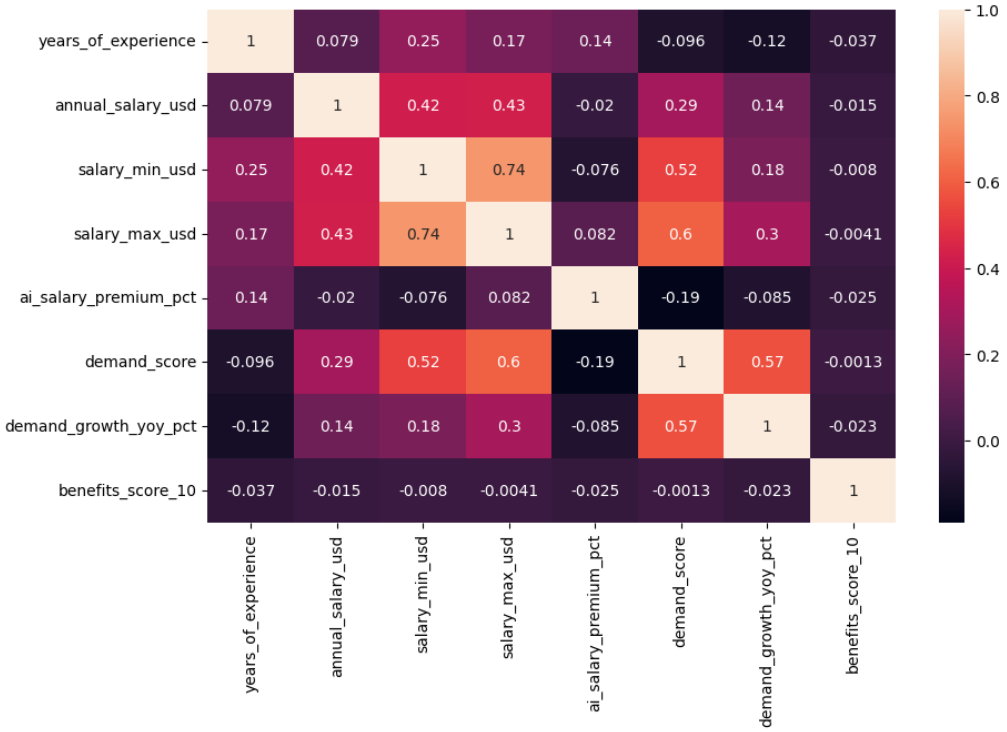



- Figure A3 illustrates the distribution of numerical variables.

## Appendix A4: Boxplots (Outliers)




## Appendix A5: Correlation Matrix




This figure displays correlations among numerical variables.

**Appendix A6: Standardization**

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df_num)
df_scaled.mean(), df_scaled.std()
```

```
(np.float64(1.4951003398285442e-16), np.float64(1.0))
```

Summary statistics after standardization, showing mean approximately zero and standard deviation equal to one.

# APPENDIX B: FULL CODES AND OUTPUTS

Google Drive Link for Codes:
*https://colab.research.google.com/drive/1hGUScZ5a0OVnSo7FzeR6BjNehBTmeK9B?usp=sharing*

```
S20851.ipynb_

from google.colab import files
uploaded = files.upload()

import pandas as pd
df = pd.read_csv("ai_jobs_market_2025_2026.csv")
df.head()
df.isnull().sum()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1500 entries, 0 to 1499
Data columns (total 25 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   job_id                 1500 non-null   object
 1   job_title              1500 non-null   object
 2   job_category           1500 non-null   object
 3   experience_level       1500 non-null   object
 4   years_of_experience    1500 non-null   int64
 5   education_required     1500 non-null   object
 6   annual_salary_usd      1500 non-null   float64
 7   salary_min_usd         1500 non-null   int64
 8   salary_max_usd         1500 non-null   int64
 9   city                   1500 non-null   object
 10  country                1500 non-null   object
 11  remote_work            1500 non-null   object
 12  company_size           1500 non-null   object
 13  industry               1500 non-null   object
 14  required_skills        1500 non-null   object
 15  ai_salary_premium_pct  1500 non-null   float64
 16  demand_score           1500 non-null   int64
 17  demand_growth_yoy_pct  1500 non-null   float64
 18  benefits_score_10      1500 non-null   float64
 19  posting_year           1500 non-null   int64
 20  posting_month          1500 non-null   int64
 21  is_senior              1500 non-null   int64
 22  is_remote_friendly     1500 non-null   int64
 23  is_llm_role            1500 non-null   int64
```

```
 24  salary_tier            1500 non-null   object
dtypes: float64(4), int64(9), object(12)
memory usage: 293.1+ KB
drop_cols = ['job_id', 'job_title', 'city', 'required_skills']
df = df.drop(columns=drop_cols)


import seaborn as sns
import matplotlib.pyplot as plt

num_cols = [
    'years_of_experience',
    'annual_salary_usd',
    'salary_min_usd',
    'salary_max_usd',
    'ai_salary_premium_pct',
    'demand_score',
    'demand_growth_yoy_pct',
    'benefits_score_10'
]

df_num = df[num_cols]

df_num.hist(figsize=(12,8))
plt.show()

    for col in df_num.columns:
        plt.figure()
        plt.boxplot(df_num[col])
        plt.title(col)
        plt.show()

import seaborn as sns
plt.figure(figsize=(10,6))
sns.heatmap(df_num.corr(), annot=True)
plt.show()

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df_scaled = scaler.fit_transform(df_num)

df_scaled.mean(), df_scaled.std()
    (np.float64(1.4951003398285442e-16), np.float64(1.0))
```

---

**PRINCIPAL COMPONENT ANALYSIS**

```
from sklearn.decomposition import PCA
import numpy as np

# Apply PCA
pca = PCA()
pca.fit(df_scaled)

# Explained variance
explained_variance = pca.explained_variance_ratio_
print(explained_variance)

    [0.34593178 0.17385033 0.1257867  0.11644089 0.09817448 0.07687324
     0.03558154 0.02736104]
```

```
import matplotlib.pyplot as plt

plt.plot(np.cumsum(explained_variance), marker='o')
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
plt.title('Scree Plot - Cumulative Variance Explained')
plt.grid()
plt.show()

import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

# Apply PCA
pca = PCA()
pca.fit(df_scaled)

# Explained variance ratio
explained_variance = pca.explained_variance_ratio_

# Plot scree plot (individual variance)
plt.figure()
plt.plot(range(1, len(explained_variance)+1), explained_variance, marker='o
')
plt.xlabel('Principal Component')
plt.ylabel('Explained Variance')
plt.title('Scree Plot (Individual Explained Variance)')
plt.show()


import numpy as np

cumulative_variance = np.cumsum(explained_variance)
plt.figure()
plt.plot(range(1, len(explained_variance)+1), explained_variance, marker='o
', label='Individual')
plt.plot(range(1, len(cumulative_variance)+1), cumulative_variance, marker=
's', label='Cumulative')

plt.xlabel('Principal Component')
plt.ylabel('Variance Explained')
plt.title('Scree Plot')
plt.legend()
plt.show()


loadings_pca = pd.DataFrame(
    pca.components_.T,
      columns=[f'PC{i+1}' for i in range(len(df_num.columns))],
    index=df_num.columns
)

print("Loadings: ",loadings_pca)
```

```
Loadings:                   PC1       PC2       PC3       PC4       PC5
years_of_experience    0.084234  0.633051  0.034292 -0.204632 -0.578763
annual_salary_usd      0.355658  0.160034  0.065965 -0.148691  0.712273
salary_min_usd         0.497029  0.223852  0.093461 -0.181276 -0.040792
salary_max_usd         0.521510  0.179328  0.022048  0.108395  0.004172
ai_salary_premium_pct -0.062631  0.456171 -0.181286  0.825429  0.155076
```

```
demand_score           0.489341 -0.303648 -0.042499  0.085783 -0.149567
demand_growth_yoy_pct  0.322312 -0.430506 -0.197391  0.361658 -0.323854
benefits_score_10     -0.011769 -0.075751  0.954787  0.268175 -0.068692

                            PC6       PC7       PC8
years_of_experience    0.421970  0.187969  0.027275
annual_salary_usd      0.554335  0.079610 -0.021866
salary_min_usd        -0.349385 -0.448093 -0.580289
salary_max_usd        -0.321219 -0.099328  0.755360
ai_salary_premium_pct -0.067060  0.080049 -0.197064
demand_score          -0.123425  0.754319 -0.229297
demand_growth_yoy_pct  0.513688 -0.415045 -0.000802
benefits_score_10      0.076137 -0.003126 -0.007898
```

**FACTOR ANALYSIS**

```
!pip install factor_analyzer
from factor_analyzer.factor_analyzer import FactorAnalyzer
from factor_analyzer.factor_analyzer import calculate_kmo, calculate_bartle
tt_sphericity

# KMO Test
kmo_all, kmo_model = calculate_kmo(df_num)
print("KMO:", kmo_model)

# Bartlett Test
chi_square_value, p_value = calculate_bartlett_sphericity(df_num)
print("Bartlett Test p-value:", p_value)
```

```
KMO: 0.6818683705817283
Bartlett Test p-value: 0.0
```

```
from factor_analyzer import FactorAnalyzer

# Fit FA without rotation
fa = FactorAnalyzer(rotation=None)
fa.fit(df_scaled)

# Get eigenvalues
eigenvalues, _ = fa.get_eigenvalues()
print("eigenvalues: ",eigenvalues)
```

```
eigenvalues:[2.76745426  1.39080264  1.00629359  0.9315271    0.78539584
0.61498589 0.28465233 0.21888835]
```

```
fa = FactorAnalyzer(n_factors=3, rotation='varimax')
fa.fit(df_scaled)

loadings_fa = pd.DataFrame(fa.loadings_,index=num_cols,
                           columns=['Factor1', 'Factor2', 'Factor3'])
print(loadings_fa)
```

```
                          Factor1   Factor2   Factor3
years_of_experience    0.293842 -0.260300  0.170154
annual_salary_usd      0.445627  0.138939  0.035016
salary_min_usd         0.922094  0.115647 -0.027706
salary_max_usd         0.778943  0.337688  0.228270
ai_salary_premium_pct -0.047478 -0.121999  0.697184
demand_score           0.452598  0.813630 -0.101749
```

```
    demand_growth_yoy_pct  0.120943  0.623185 -0.003594
    benefits_score_10     -0.011453 -0.002456 -0.040764
plt.figure()
plt.plot(range(1, len(eigenvalues)+1), eigenvalues, marker='o')
plt.axhline(y=1, linestyle='--')
plt.xlabel('Factor Number')
plt.ylabel('Eigenvalue')
plt.title('Scree Plot (Factor Analysis)')
plt.show()
```

---

**DISCRIMINANT ANALYSIS**

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix

# Features (numerical variables)
X = df_num

# Target variable
y = df['salary_tier']

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Fit LDA model
lda = LinearDiscriminantAnalysis()
lda.fit(X_train, y_train)

# Predictions
y_pred = lda.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
Accuracy: 0.9355555555555556
[[ 35   0   0   0   0]
 [  0  19   4   0   0]
 [  0   4 115   0   1]
 [  7   0   0 138   8]
 [  0   0   5   0 114]]
```

---

```
!pip install bioinfokit
from bioinfokit.analys import stat

# Prepare dataset (numerical + group)
data = df_num.copy()
data['group'] = df['salary_tier']   # target variable

# Run Box's M test
res = stat()
```

```
if hasattr(res, 'box_m'):
    res.box_m(df=data, grp_col='group')
    print(res.box_m_test)
else:
    print("Error: The 'stat' object from bioinfokit.analys does not have a
'box_m' method.")


from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis

# Initialize and fit QDA model
qda = QuadraticDiscriminantAnalysis()
qda.fit(X_train, y_train)

# Predictions
y_pred_qda = qda.predict(X_test)

# Accuracy
accuracy_qda = accuracy_score(y_test, y_pred_qda)
print(f"QDA Accuracy: {accuracy_qda}")

# Confusion matrix
cm_qda = confusion_matrix(y_test, y_pred_qda)
print("QDA Confusion Matrix:\n", cm_qda)
```

```
QDA Accuracy: 0.9422222222222222
QDA Confusion Matrix:
 [[ 34   0   0   1   0]
 [  0  23   0   0   0]
 [  0   0 112   0   8]
 [  4   0   0 146   3]
 [  0   0   7   3 109]]
```

```
from sklearn.metrics import f1_score, recall_score, classification_report

# Calculate and print F1-score and Recall for LDA

f1_lda = f1_score(y_test, y_pred, average='weighted')
recall_lda = recall_score(y_test, y_pred, average='weighted')
print(f"LDA F1-score (weighted): {f1_lda:.4f}")
print(f"LDA Recall (weighted): {recall_lda:.4f}")
```

```
LDA F1-score (weighted): 0.9359
LDA Recall (weighted): 0.9356
```

```
# Calculate and print F1-score and Recall for QDA

f1_qda = f1_score(y_test, y_pred_qda, average='weighted')
recall_qda = recall_score(y_test, y_pred_qda, average='weighted')
print(f"QDA F1-score (weighted): {f1_qda:.4f}")
print(f"QDA Recall (weighted): {recall_qda:.4f}")
```

```
QDA F1-score (weighted): 0.9424
QDA Recall (weighted): 0.9422
```

```
# Print classification report for LDA
print("\nLDA Classification Report:\n")
print(classification_report(y_test, y_pred))
```

```
LDA Classification Report:

                       precision    recall  f1-score   support

       Elite (>$300k)       0.83      1.00      0.91        35
       Entry (<$100k)       0.83      0.83      0.83        23
      Mid ($100-150k)       0.93      0.96      0.94       120
   Senior ($200-300k)       1.00      0.90      0.95       153
Upper-Mid ($150-200k)       0.93      0.96      0.94       119

            accuracy                           0.94       450
           macro avg       0.90      0.93      0.91       450
        weighted avg       0.94      0.94      0.94       450
```

```
QDA Classification Report:

                       precision    recall  f1-score   support

       Elite (>$300k)       0.89      0.97      0.93        35
       Entry (<$100k)       1.00      1.00      1.00        23
      Mid ($100-150k)       0.94      0.93      0.94       120
   Senior ($200-300k)       0.97      0.95      0.96       153
Upper-Mid ($150-200k)       0.91      0.92      0.91       119

            accuracy                           0.94       450
           macro avg       0.94      0.95      0.95       450
        weighted avg       0.94      0.94      0.94       450
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# Set up the matplotlib figure and axes for two subplots
fig, axes = plt.subplots(1, 2, figsize=(16, 6))


# Plot Confusion Matrix for LDA
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=lda.classes_, yticklabels=lda.classes_)
axes[0].set_title('LDA Confusion Matrix')
axes[0].set_xlabel('Predicted Label')
axes[0].set_ylabel('True Label')


# Plot Confusion Matrix for QDA
sns.heatmap(cm_qda, annot=True, fmt='d', cmap='Greens', ax=axes[1],
            xticklabels=qda.classes_, yticklabels=qda.classes_)
axes[1].set_title('QDA Confusion Matrix')
axes[1].set_xlabel('Predicted Label')
axes[1].set_ylabel('True Label')

plt.tight_layout()
plt.show()
```